AUTOMATED QUESTION-AWARE BUDGET-CONSTRAINED

VideoAgent

Question-Aware Cascaded Video Analysis for Objective Multiple-Choice QA

KRAFTON AI R&D Hackathon · Round 2 · Problem 3 · Spring 2026

20	≤ 1200s	≤ 15 min	20 letters
video–prompt pairs per test run	maximum duration per video	hard wall-clock submission budget	exact output format guaranteed

Core idea. VideoAgent does not treat this challenge as generic video summarization. It treats it as **prompt-conditioned evidence retrieval** under a strict wall-clock budget. The system first interprets the question, then allocates computation only where the question demands it: audio-localized frame sniping for sound-triggered queries, scout-pass frame sampling for simple visual questions, and full-video analysis for hard visual or temporal reasoning.

Executive Summary

We built an end-to-end agent that automatically answers objective multiple-choice questions about videos with no manual intervention at test time. The design goal was to maximize accuracy while still being robust to long videos, API latency, and output-format failures.

What is novel in this system

- **Question-first routing** before expensive video processing.
- **Option-aware verification** instead of free-form answer generation.
- **Structured JSON output** to constrain answers to valid option letters.
- **Wall-clock safety mechanisms** to guarantee a complete 20-letter submission.

Why this design fits the challenge

- Not all questions need the same evidence granularity.
- Long videos punish naive full-video prompting on every sample.
- Objective multiple-choice QA benefits from explicit option verification.
- Hackathon conditions reward robust orchestration as much as model quality.

1. System Overview

The pipeline starts from the question, not the pixels. We first parse the prompt, extract all answer choices dynamically, classify the reasoning type, and choose the cheapest evidence acquisition strategy that is still likely to answer the question correctly. This is the main departure from a single end-to-end “watch the whole video and answer” baseline.

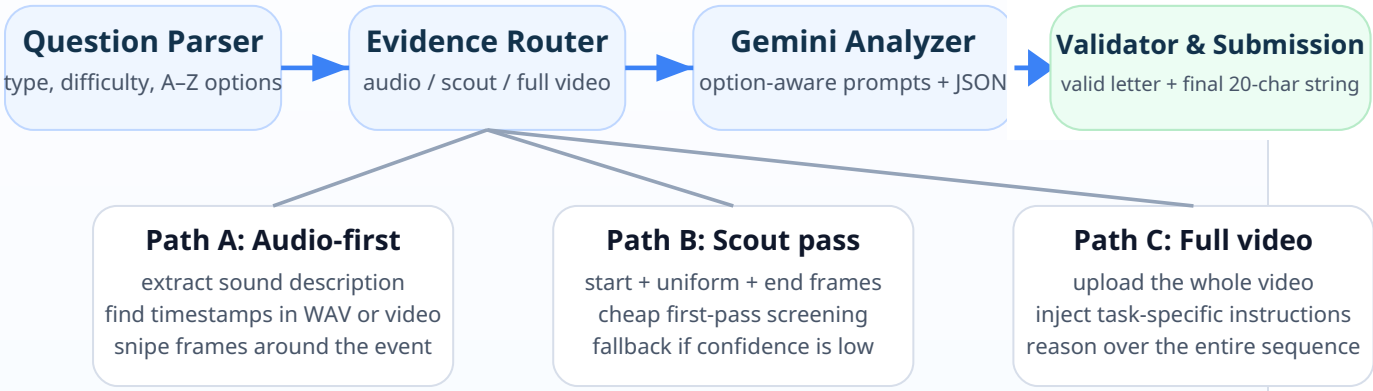

Figure 1. High-level architecture. The router spends expensive full-video compute only when the question demands it.

2. Question-Aware Routing

Routing policy is the most important systems decision in this project. A single fixed analysis mode is either too slow or too brittle. Our router combines **question type**, **difficulty heuristics**, **video duration**, and **upload-size limits** to select a path.

Route	Typical triggers	Evidence strategy	Why it helps
Audio-first	sound, hear, horn, voice, music	Extract the target sound description, search timestamps in a WAV file or directly in the video audio track, then analyze frames around the detected moment. Retry with denser frame sampling if confidence is low.	Most audio questions are really “what is visible at the moment of this sound?” Timestamp localization narrows the search space dramatically.
Hard visual / temporal	brand, logo, text, scene count, first / fifth / last	Skip scout pass and send the full video with specialized instructions for OCR/brand identification, ordered-event tracking, or scene counting.	These questions depend on global chronology or fine-grained evidence that sparse frame sampling often misses.
Easy visual	Simple presence, color, brightness, or coarse motion questions on short videos	Scout pass uses one start frame, uniform frames across the clip, and end frames. Accept if confidence ≥ 0.85; otherwise escalate to full-video analysis.	Saves time and tokens on easy cases while preserving an escalation path for uncertain predictions.
Large-file fallback	Video size > 200 MB	Avoid upload bottlenecks and use uniform frame extraction instead.	Prevents large uploads from dominating the wall-clock budget.

Implementation detail. The current code uses three coarse question classes: **A** (audio), **B** (visual), and **C** (temporal/order-sensitive). Additional hard-keyword rules decide whether scout pass can be skipped safely.

3. Evidence Acquisition and Task-Specific Prompting

Scout pass

Scout pass is intentionally cheap. For very short clips, it samples **9 frames** (start + 6 uniform + 2 end). For longer short clips, it samples **19 frames** (start + 15 uniform + 3 end), deduplicates nearby timestamps, and asks the model to answer using only these frames.

This pass is effective when the answer is persistent or visually obvious, but it is not trusted on sparse-event or order-sensitive questions.

Audio-localized frame sniping

For audio questions, the system first searches for the exact sound event and then extracts consecutive frames around the detected timestamp. This converts a broad multimodal reasoning problem into a much narrower visual QA problem centered on the relevant moment.

If the first answer has confidence below **0.70**, the system retries using denser 0.2-second frame spacing.

Full-video prompt specialization

Full-video prompts are not generic. The router injects task-specific instructions that match the failure mode of the question:

- **brand_ocr**: quote exact text/logo and describe location on the object.
- **counting_scene**: merge minor zoom/pan changes and list unique viewpoints before counting.
- **ordered_event**: enumerate every qualifying event chronologically, then select the Nth one.
- **audio_trigger**: anchor the answer to the precise sound timestamp.

Why prompting matters here

In this hackathon setting, carefully engineered prompts delivered more value than hurriedly stacking additional specialist modules. The prompts explicitly force the model to **list evidence before answering**, which reduces superficial guesses and makes reasoning more aligned with the objective nature of the benchmark.

The codebase stores these templates centrally in `prompts.py`, which keeps the routing logic simple while still allowing different reasoning behaviors per question type.

4. Option-Aware Answering and Output Reliability

The challenge requires a single answer letter for each sample, not a free-form explanation. We therefore model the task as **verification over a finite option set** instead of unconstrained generation.

Dynamic option parsing

The system extracts answer choices dynamically from each prompt, supporting variable option counts from **A through Z**. This matters because the benchmark does not guarantee a fixed 4-way format.

Every model response is validated against the parsed option set. Invalid outputs are replaced with a safe fallback letter so the final answer string is always well-formed.

Structured JSON output

Gemini is asked to return a strict JSON object with `answer`, `confidence`, and `reasoning`. The answer field is constrained to the valid option letters for that question.

```
{ "answer": "C", "confidence":
0.82, "reasoning": "The relevant
event appears after the horn at
around 12.5s..." }
```

Important empirical finding. In our testing, raw VLM confidence was often poorly calibrated: answers could be incorrect even when the model reported very high confidence. For that reason, confidence is used mainly for **routing decisions** (e.g., scout accept / retry) rather than as a direct proxy for correctness.

5. Time-Budget Engineering

The benchmark is as much a systems problem as a reasoning problem. A strong answering strategy that misses the deadline is still a failed submission. VideoAgent therefore implements multiple layers of time control.

Layer	Mechanism	Current setting in code
Parallelism	All samples are processed concurrently using a thread pool.	`ThreadPoolExecutor(max_workers=10)`
Per-video timeout	Each sample has its own timeout. If a thread finishes late but already produced a partial result, that result can still be used.	`180s` (Flash) / `240s` (Pro)
Internal wall-clock budget	The system targets a 14-minute internal budget to leave extra time for final packaging and submission.	`MAX_TOTAL_TIME = 14 min`
Global fallback trigger	If the run is already late, remaining items are forced through a cheaper fallback path.	`FALLBACK_TIME = 12 min`
Output guarantee	The validator always emits exactly *N* letters in index order, filling missing entries with a default safe answer if necessary.	`validate_final_output()`

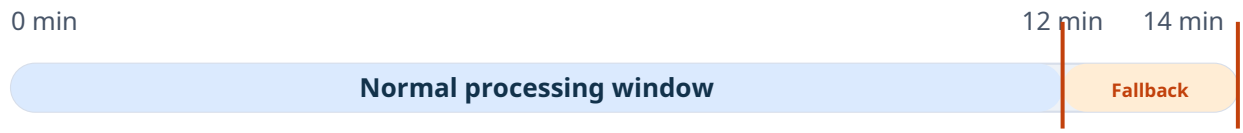

Figure 2. Time-budget policy. The implementation intentionally leaves slack instead of spending the full 15 minutes on inference.

Mode	Model	Passes	Use case
`(default)`	Gemini 3 Flash preview	1	Fastest mode for deadline-sensitive evaluation.
`--pro`	Gemini 3.1 Pro preview	1	Higher accuracy on difficult reasoning, but higher timeout risk.
`--vote`	Flash	3	Majority vote across repeated runs when extra time is available.
`--pro --vote`	Pro	3	Most expensive mode; mainly useful for offline testing.

6. What We Learned During Development

Confidence is not enough

Repeated runs on difficult questions often produced different answers with similarly high confidence values. This pushed us away from confidence-based retries and toward question-pattern-based routing.

Scene counting remains difficult

Counting distinct camera angles or viewpoints is one of the hardest categories for current VLMs. Models tend to over-split minor framing changes into new scenes.

Ordered events need global chronology

Questions such as “what is the fifth close-up instrument?” require exhaustive chronological enumeration. Full-video analysis plus ordered-event prompting was substantially safer than sparse sampling.

Scout pass is high leverage but brittle

Uniform frame sampling works very well for persistent properties like color or presence, but it can miss sparse events entirely in long videos. Restricting scout pass to low-complexity questions was essential.

7. Limitations and Extensions

Current limitation	Likely next improvement
Distinct scene / angle counting is unstable.	Add shot-boundary detection plus perceptual-hash or embedding-based viewpoint clustering.
Object counts can still drift on crowded scenes.	Integrate detector + tracker pipelines such as YOLO + ByteTrack/ BoT-SORT for explicit counting.
Ordered-event questions remain expensive.	Use shot segmentation and higher-FPS localized classification only on candidate windows.
Majority vote improves stability but is costly.	Explore cheaper self-consistency schemes, temperature control, or route-specific ensembles.

8. Implementation Notes

Rather than emphasizing repository structure, the implementation is described here in terms of functional stages that directly support the competition goal: parse the question, gather the cheapest useful evidence, verify against the explicit options, and guarantee a complete 20-letter submission within the deadline.

Functional area	Role in the pipeline
Question interpretation	Extract the question type, difficulty cues, and full answer-choice set before expensive analysis begins.
Evidence routing	Choose among audio-first lookup, scout-pass sampling, or full-video reasoning depending on the evidence required.
Audio localization	Narrow sound-triggered questions to a small time window so the visual search space becomes manageable.
Frame sampling	Use sparse sampling for easy visual questions and denser localized extraction when sparse evidence is insufficient.
Option-aware verification	Constrain outputs to valid option letters and treat the task as verification over explicit choices, not free-form generation.
Submission safety	Use timeouts, fallback paths, and final validation so every item still contributes a valid answer letter before the deadline.

Technology profile

Primary VLM	Gemini family models
Pre-processing	Audio lookup + frame extraction
Video evidence	Scout sampling and full-video analysis
Parallelism	Concurrent multi-sample processing
Output	Structured option-constrained responses

Submission-oriented design

- No manual intervention after the run starts.
- Question format and option count are treated as variable.
- Hard cases receive more budget; easy cases stay cheap.
- Late or uncertain items fall back to safer lightweight paths.
- Final validation enforces a complete answer string.

Closing Remark

VideoAgent is a pragmatic hackathon system: small enough to build quickly, but structured enough to handle the core difficulty of the benchmark. The report’s central claim is simple: **accurate video QA under a hard deadline requires question-aware allocation of reasoning budget.** Our implementation operationalizes that claim through routing, constrained prompting, and system-level safeguards that convert a general-purpose VLM into a more reliable competition-time agent.